

タイトル: Log4Shell (JVNDB-2021-005429・CVE-2021-44228 など) 脆弱性に関する調査レポート

1. はじめに

ソフトウェアは現代社会を支える基盤であり、それに伴い広く展開された実装に影響を与える新たな脆弱性の脅威が絶えず存在している。過去には、Heartbleed (2014 年)、Rowhammer (2015 年)、Spectre (2018 年) [といった脆弱性が世界中のシステム運用に重大な影響を及ぼしてきた。そして、これらに続く最新の重大な脆弱性が「Log4Shell (2021 年)」である。Log4Shell は、広く使用されている Java ログインライブラリ「Log4j」に細工された文字列を挿入することにより、脆弱なアプリケーションを通じてリモートコード実行を可能にするものである。このような脆弱性が公表された直後から、攻撃者と防御側との間で“競争”が始まる。攻撃者は、システムが修正される前に脆弱性を悪用しようとし、システム管理者や企業はそれに先んじて修正パッチを適用し、被害を防ぐ必要がある。Log4j は Java アプリケーションの中核的な要素であるため、その脆弱性 (Log4Shell) は、アプリケーションの実装によって多様な方法で悪用される可能性がある。[1]

レポートでは、これらの脆弱性 (CVE-2021-44228 など) について個別に整理し、その影響範囲、原因、対策を明らかにする。(CVE - Common Vulnerabilities and Exposures) JVNDB-2021-005429・CVE-2021-44228・CVE-2021-45046・CVE-2021-45105・CVE-2021-44832

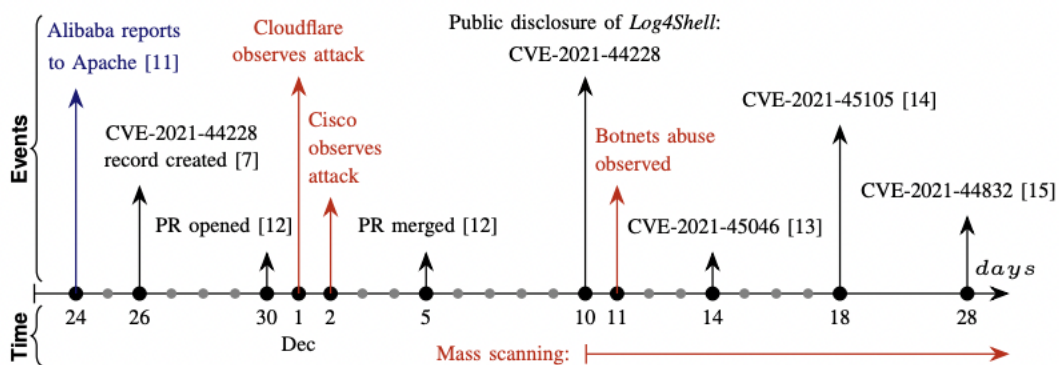


図 1: Log4Shell 脆弱性の報告から 2021 年の結果までの展開。[2]

1. JVNDB-2021-005429

JVND-2021-005429 は Apache HTTP Server 2.4.49 に存在するディレクトリ・トラバーサル脆弱性であり、細工されたリクエストを送信することで、本来アクセスできないファイルにアクセスされる可能性があるという深刻な問題である。特にこの脆弱性は、リモートの第三者によって任意のファイルが読み取られる可能性があるため、設定ファイルやパスワードファイルといった重要な情報の漏洩につながる恐れがあり、機密性の高いシステムでは致命的な影響を与えることがある。興味深い点として、この脆弱性はバージョン 2.4.49 におけるアクセス制御の実装における設計ミスから生じており、URL パスの正規化が不完全であったことが主な原因である。実際にこの脆弱性を利用した攻撃の PoC（概念実証コード）もすぐに公開され、多くの攻撃者によって悪用されたことから、ゼロデイに近い形で広く拡散された事例としても注目されている。その Lookup 機能の内、JNDI Lookup 機能を悪用することにより、ログに含まれる外部の URL もしくは内部パスから Java のクラス情報をデシリアライズして実行してしまう問題（CWE-20, CVE-2021-44228）が発見されました。これにより、遠隔の攻撃者が細工した文字列を脆弱なシステムのログに記載させ、結果として任意の Java コードをシステムに実行させることが可能です。[4]

2. CVE-2021-44228 (Log4Shell)

公開日：2021 年 12 月 9 日/ 原因：JNDI Lookup 機能が外部入力を許可していた。

対策：Log4j 2.15.0 以降へのアップデート／lookup 機能無効化

Apache Log4j2 のバージョン 2.0-beta9 から 2.15.0（ただし、セキュリティパッチが適用された 2.12.2、2.12.3、および 2.3.1 を除く）において、設定ファイル、ログメッセージ、およびログパラメータで使用する JNDI（Java Naming and Directory Interface）機能に対する適切な保護が施されていなかったため、攻撃者が制御する LDAP サーバなどの JNDI エンドポイントを通じて、任意のコードが実行される危険性がある重大な脆弱性（CVE-2021-44228）が存在していた。攻撃者は、ログメッセージやログパラメータに特殊な文字列（例：\${jndi:ldap://...}）を埋め込むことで、外部の LDAP サーバから任意の Java クラスを読み込み、リモートでコードを実行することが可能であった。この問題に対し、バージョン 2.15.0 以降では JNDI を使用したメッセージルックアップ機能がデフォルトで無効化され、さらに 2.16.0 以降（および 2.12.2、2.12.3、2.3.1 では）JNDI に関する機能自体が完全に削除された点により、根本的な対策が取られた。本脆弱性は log4j-core モジュールに特有であり、Apache の他のロギングプロジェクト（log4net、log4cxx など）には影響しないことにも留意が必要である。[2]

3. CVE-2021-45046 (44228 の対策が不完全で、特定の構成では再度 RCE が可能)

CVE-2021-45046 は、CVE-2021-44228（通称 Log4Shell）に対する Apache Log4j 2.15.0 の修正が一部の非デフォルト設定において不完全であったことにより、新たに発見された脆弱性である。攻撃者が特定のパターンレイアウト設定において JNDI ルックアップ構文を利用した悪意のある入力をログに含めることで、特定環境において情報漏洩やリモートコード実行（RCE）、全環境においてローカルコード実行（LCE）が可能となるリスクがあるとされた。この問題は、Log4j 2.16.0 にて JNDI の機能を完全に削除することで根本的に解消された。[3]

4. CVE-2021-45105

この脆弱性は「Log4Shell」（CVE-2021-44228）の直接的な亜種ではないものの、同種の攻撃経路であり、攻撃者が制御するログ情報の Lookup 機能を悪用する点で両者は類似しています。ただしこの脆弱性の場合、JNDI 以外の Lookup が悪用される可能性があります。Apache Log4j の API は、Lookup における変数の置換をサポートしています。この際、細工された変数を用いることで、制御不能な再帰的な置換により、アプリケーションをクラッシュさせることが可能になります。つまりこれにより、Lookup のコマンドを制御できる攻撃者（例：Thread Context Map 経由など）は不正な Lookup 変数を細工し、DoS 攻撃を引き起こすことができます。この現象は、バージョン 2.16.0 を含む Log4j の各バージョンのテストで確認されています。

■ どのような脆弱性か

StrSubstitutor クラスでネストした変数を置換する場合、substitutor() クラスを再帰的に呼び出します。しかし、ネストされた変数自体が置換対象の変数として参照される場合、同じ文字列が再帰的に呼び出されることになり、無限再帰が発生し、サーバへサービス拒否攻撃を実行する状態となってしまいます。例えば、パターンレイアウトに `${ctx.apiversion}` の Context Lookup があり、その置換値が `${${ctx.apiversion}}` であれば、その変数は再帰的に自身を置き換える状態となってしまいます。

5. CVE-2021-44832

Apache Log4j において、新たな脆弱性 CVE-2021-44832 が報告された。この脆弱性は、攻撃者がログ構成ファイルを変更できる権限を持っている場合に、JNDI URI を参照するデータソースを用いた JDBC Appender を悪用し、任意のコードをリモートで実行できる可能性がある。この脆弱性の CVSS（共通脆弱性評価システム）による深刻度スコアは 6.6 と評価されており、「中程度（Medium）」に分類されている。影響を受ける環境は限定的であるが、ログ設定ファイルの改ざんが可能な場合、攻撃者はログ構成に悪意のある設定を追加することで、特権の昇格やリモートコード実行（RCE）が可能となる。本脆弱性は、Apache Log4j の他の脆弱性（CVE-2021-44228 など）と同様に、JNDI の機能を介したリモートコード実行に関するものであり、システム構成や設定によっては深刻な被害を招く可能性がある。これにより、Log4j を利用するシステムに対しては、最新版へのアップデートや、ログ設定の権限管理を強化するなど、早急な対策が求められる。[1]

まとめ

今回の調査を通じて、クリティカルな脆弱性（重大なセキュリティホール）は、単一の要因ではなく、複数の小さなミスや設計上の欠陥が組み合わさることによって発生することが多いという点を再認識しました。特に、複数の脆弱性が連鎖的に悪用される「チェーン型攻撃（Exploit Chain）」が RCE（リモートコード実行）に至るまでの典型的なパターンであることが分かりました。また、脆弱性を表面的に見るのではなく、実際の攻撃手法やコードレベルでその仕組みを一つ一つ丁寧に追っていくことで、単に「危険」とされるだけでなく、「なぜ危険なのか」「どのように防ぐべきか」といった本質的な理解が深まったと感じています。さらに、本脆弱性の調査を通して、セキュリティ分野における予防的な視点の重要性、つまり脆弱性が発見される前に設計段階から安全性を意識するソフトウェア開発（Secure Coding）やセキュア設計の重要性にも改めて気づかされました。

参考文献：

- [1] The Race to the Vulnerable: Measuring the Log4j Shell Incident (<https://arxiv.org/pdf/2205.02544>)
- [2] (<https://nvd.nist.gov/vuln/detail/cve-2021-44228>)
- [3] (<https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2021-45046>)
- [4] <https://jvndb.jvn.jp/ja/contents/2021/JVNDB-2021-005429.html>